Jenkinsfile – Nodos y Pipeline

Alumno: Yamil Motta

Asignatura: Proyecto Final de Grado

Ciclo: 2ºASIR

Curso: 25/26

Nodos en Jenkins (Agent) 3

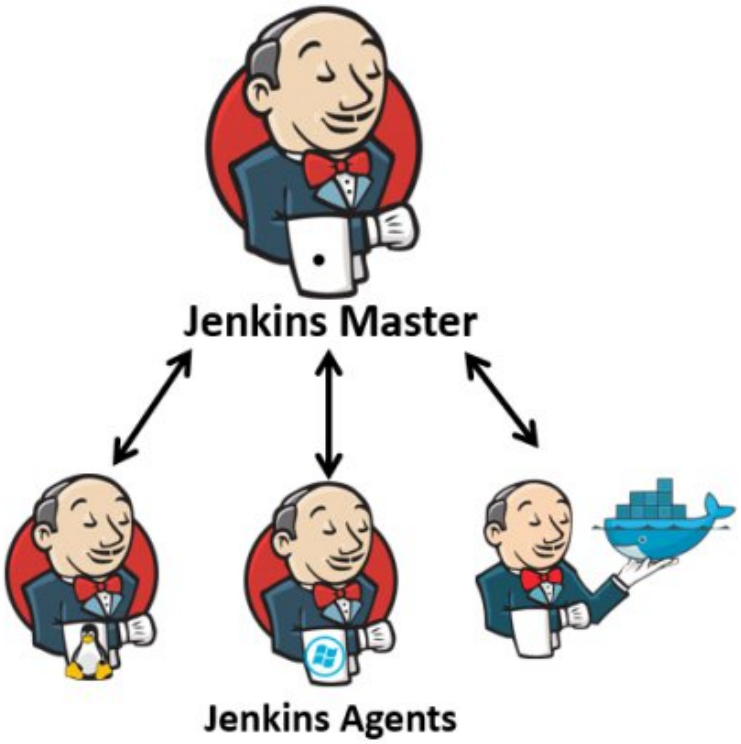
¿Qué es un Nodo? 3
Built-in Node 3
De Nodo estático a dinámico 4
Objetivos 5
Jenkinsfile y Pipeline 6

Jenkinsfile 6
Estructura de un Jenkinsfile 6
Configuración de Pipeline 7
Agentes Dinámicos como Código 8
Plugin Docker para el Agente Dinámico 9
Primera Pipeline de Integración Continua (CI) 10

Nodos en Jenkins (Agents)

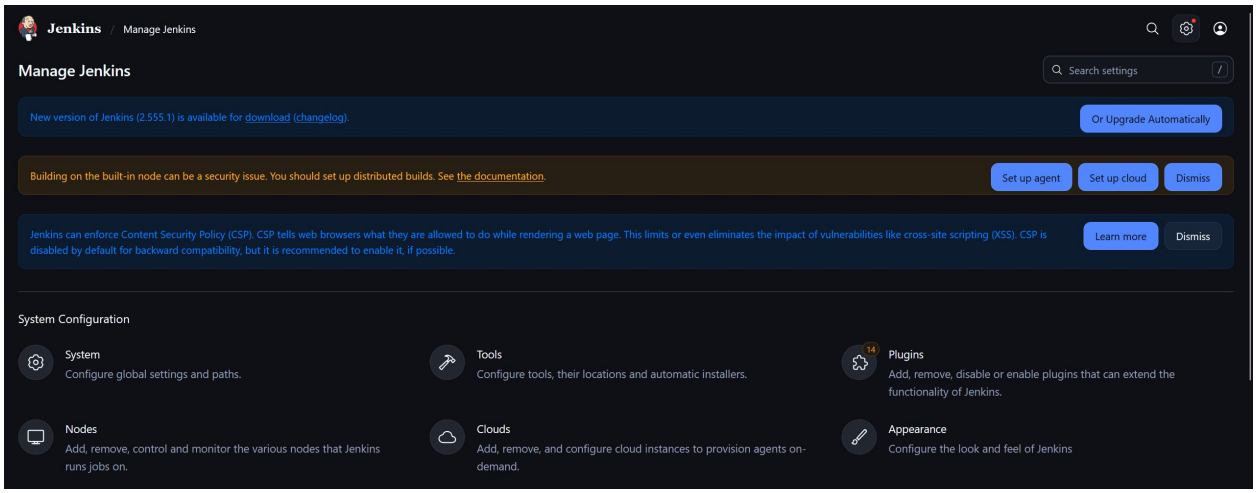
¿Qué es un Nodo?

Un nodo en Jenkins es cualquier máquina física o virtual que Jenkins utiliza para ejecutar trabajos, incluyendo el nodo maestro (ahora llamado controlador) y los nodos agentes que se conectan al maestro para realizar tareas. Estos nodos permiten distribuir la carga de trabajo y ejecutar trabajos en diferentes entornos y configuraciones. Los nodos de Jenkins también se caracterizan por formar parte fundamental de la sintaxis de script del recurso de la pipeline cuando los nodos se encuentran en bloques.



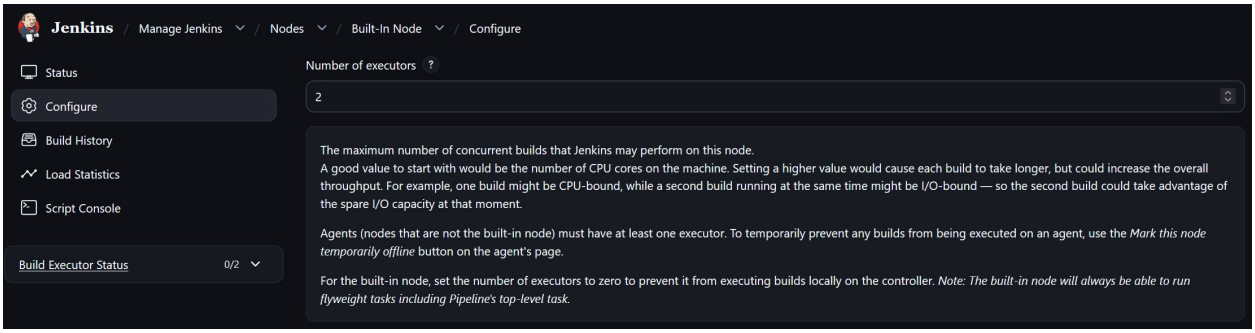
Built-in Node

Por defecto, Jenkins ejecuta sus Nodos (o Agents) con los máximos privilegios y de forma estática. A pesar de ello, avisa que es una práctica no recomendada y poco segura en el panel web:



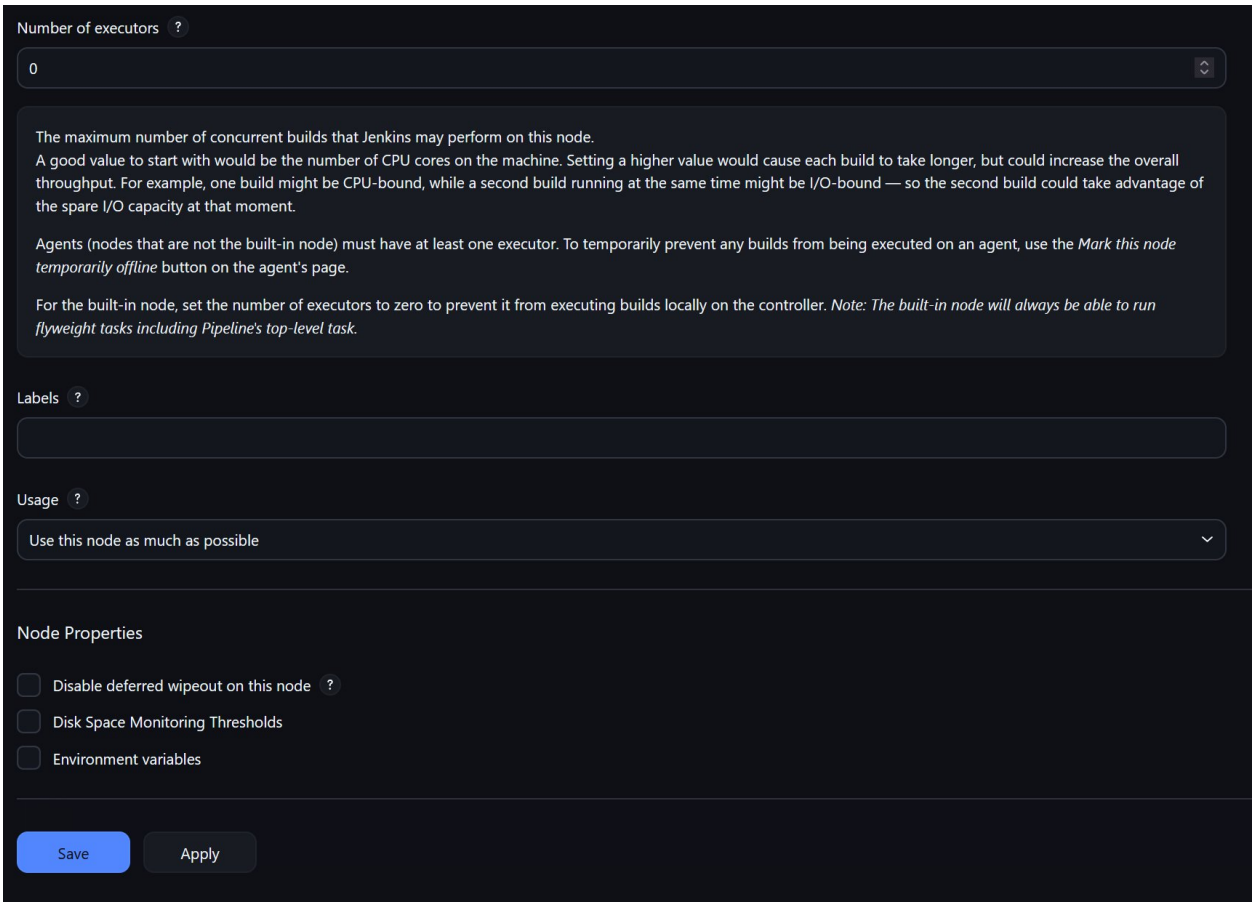
Si ejecutamos los pipelines (compilar código, lanzar scripts) directamente en el nodo maestro de Jenkins, y un desarrollador mete un código malicioso (o un atacante hace un Pull Request), ese código se ejecuta con los permisos del servidor principal. Podrían robar las credenciales de AWS o tomar el control de la máquina virtual. Por ello, cambiamos el Nodo “Built-in” que por defecto, tiene 2 ejecutores (capacidad de Agentes/Nodos estáticos) a 0.

En Manage Jenkins > Nodes > Built-in Node > Configure:



De Nodo estático a dinámico

El objetivo es hacer Nodos/Agentes efímeros ejecutándose en Docker. Teniendo en cuenta que ejecutamos Jenkins en Docker (y configuramos el docker.sock), cuando Jenkins reciba código nuevo, levantará un contenedor Docker nuevo, limpio y aislado (el Agente/Nodo), ejecutará el pipeline ahí dentro y, cuando termine, destruirá el contenedor.



Un ataque de inyección de comandos en el código fuente o dependencias maliciosas (Supply Chain Attack) podría derivar en una Ejecución Remota de Código con privilegios sobre el volumen de Jenkins y la bóveda de credenciales principales.Con el socket de Docker (DooD) configurado en fases previas, el sistema se ha diseñado para levantar contenedores aislados de un solo uso por cada ejecución de la pipeline. Esto nos da la inmutabilidad del entorno de construcción, previene la persistencia de malware post-ejecución y aísla los recursos del servidor maestro.

Objetivos

Al diseñar la topología CI/CD (Jenkinsfile), tomamos en cuenta vectores de ataque comunes en entornos de empresa (como los expuestos en plataformas de simulación como TryHackMe). En arquitecturas tradicionales, es común observar varios agentes estáticos (como GitLab Runners o Jenkins Agents) segmentados por nivel de privilegio a nivel de red, donde solo un agente específico posee visibilidad hacia los entornos de Desarrollo y Producción. El riesgo de este modelo está en la persistencia: si el nodo con privilegios de despliegue es comprometido mediante una inyección de código en el pipeline, el atacante obtiene una cabeza de puente permanente hacia la infraestructura crítica.



Ejemplo de red de pipelines, JAgent (Nodo de Jenkins) y GRunner (Agente de GitLab) conviviendo en una misma red. Este ejemplo pertenece a la Room CI/CD Build and Security.

Para mitigar este riesgo de movimiento lateral en nuestro Proyecto, cambiamos la segmentación de red estática por un modelo de Agentes Efímeros. Al utilizar contenedores Docker de un solo uso. Al finalizar el despliegue, el contenedor se elimina.

Los datos sensibles (claves, tokens) se gestionan mediante el inyector de credenciales de Jenkins, que los expone como variables de entorno efímeras en la memoria del contenedor. Las variables de configuración de la aplicación y rutas de despliegue se declaran explícitamente en el bloque environment del propio Jenkinsfile, lo que permite el control de versiones en GitHub y la “portabilidad” de la pipeline.

Jenkinsfile y Pipeline

Jenkinsfile

Un Jenkinsfile es un archivo de configuración que define todo el pipeline de CI/CD en Jenkins. Este archivo se almacena en el repositorio del proyecto y permite automatizar el proceso de construcción, prueba y despliegue del software. El Jenkinsfile de nuestro proyecto se ubica en el branch main/ de nuestro repositorio de Proyecto Final en Github.

Ventajas del uso de Jenkinsfile:

- Control de Cambios: Los cambios en el pipeline se rastrean en Git.
- Reproducibilidad: Cualquier persona que clone el repositorio puede ejecutar el mismo pipeline.
- Automatización: No es necesario configurar trabajos manualmente en la interfaz de usuario de Jenkins.

Estructura de un Jenkinsfile

Un Jenkinsfile puede contener diferentes secciones, incluyendo:

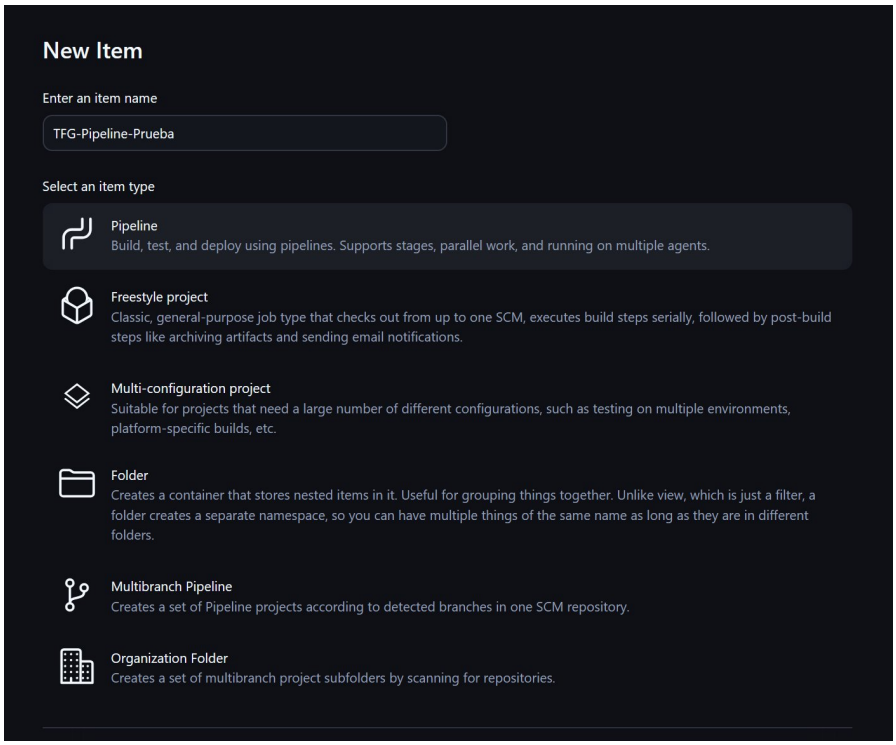
- Pasos de Construcción: Instrucciones sobre cómo compilar el código.
- Pasos de Prueba: Instrucciones para ejecutar pruebas sobre el código.
- Pasos de Despliegue: Instrucciones para desplegar el software en un entorno específico.

Ejemplo Básico

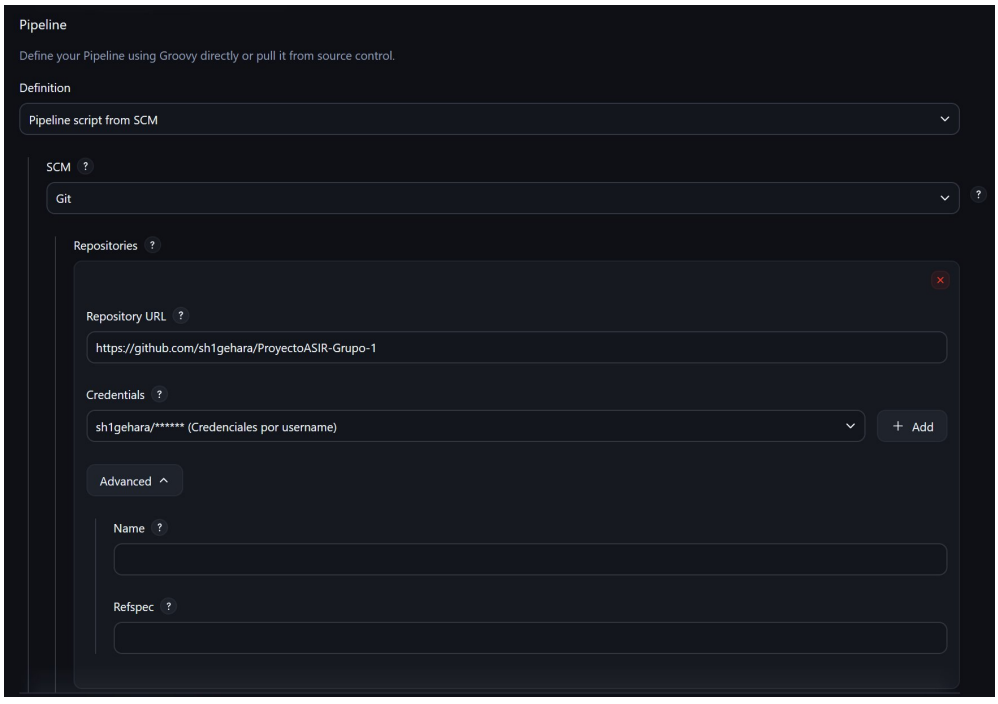
```
pipeline { //Inicio de la pipeline
    agent any   //Especificamos qué agente usar (si está disponible)
    stages { //”Pasos” que sigue la pipeline en sí.
        stage('Build') {
            steps { //Aquí dentro va el script a ejecutar en los diferentes stages
                echo 'Building...'
            }
        }
        stage('Test') {
            steps {
                echo 'Testing...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying...' //Dependiendo del entorno (Desarrollo o Main), el Deployment se hace a un entorno
de staging/qa o Producción
            }
        }
    }
}
```

Configuración de Pipeline

En Jenkins > New item:



Al crear el Item, nos deja un menú de configuración de la pipeline:



La pipeline va por Script, por SCM (lo busca en el repositorio de Github) le pongo el URL y las credenciales de mi Usuario, que tiene permisos de lectura y escritura en el repositorio. La branch es “main”, ya que solo disponemos de ella.

Agentes Dinámicos como Código

Uno de los retos de diseño en la integración continua (CI) es la gestión del entorno donde se ejecutan las pruebas. En arquitecturas tradicionales, esto requiere montar y mantener servidores dedicados, lo que genera problemas de dependencias residuales ("funciona en mi máquina") y un mayor consumo de recursos.

Para nuestra arquitectura, decidimos prescindir de agentes estáticos (los predeterminados). Aprovechando el montaje del socket de Docker configurado en la fase de infraestructura, hacemos la creación de los agentes en el propio código del pipeline (Jenkinsfile).

Una primera Pipeline (CI) de prueba es la subida a continuación:

sh1gehara Jenkins pipeline

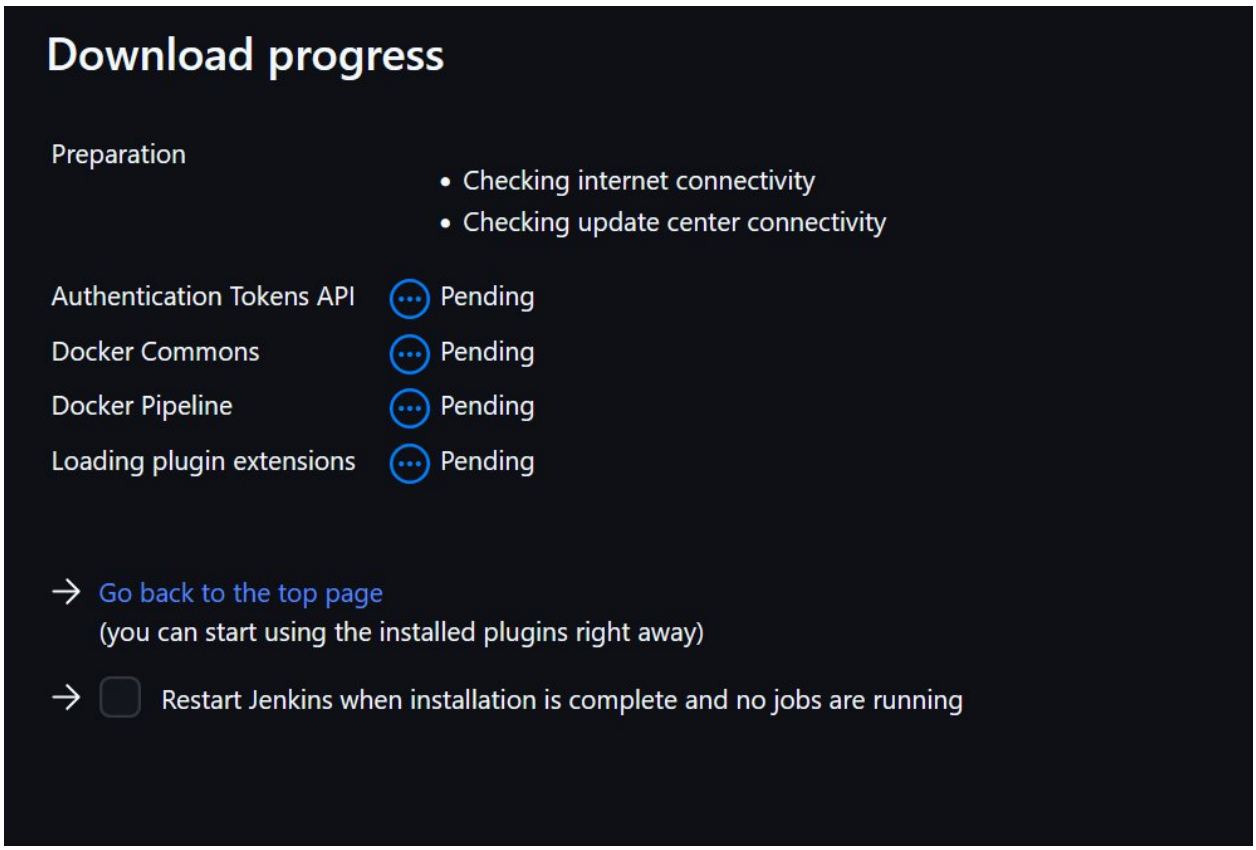
Code Blame 30 lines (28 loc) · 881 Bytes

```
1   pipeline {
2       agent {
3           docker {
4               image 'php:8.2-cli'
5               args '-u root'
6           }
7       }
8       stages {
9           stage('Descargar Código') {
10              steps {
11                  echo 'Clonando el repositorio ProyectoASIR-Grupo-1...'
12                  checkout scm
13              }
14          }
15          stage('Análisis de Código (Linter PHP)') {
16              steps {
17                  echo 'Ejecutando el runner efímero (Contenedor PHP)...'
18                  sh 'php -v'
19
20                  echo 'Verificando el directorio de la app de Damas:'
21                  sh 'ls -la app/damas/'
22
23                  echo 'Validando sintaxis estática de los archivos backend...'
24                  sh 'php -l app/damas/api.php'
25                  sh 'php -l app/damas/GameLogic.php'
26                  sh 'php -l app/damas/DB.php'
27              }
28          }
29      }
30  }
```

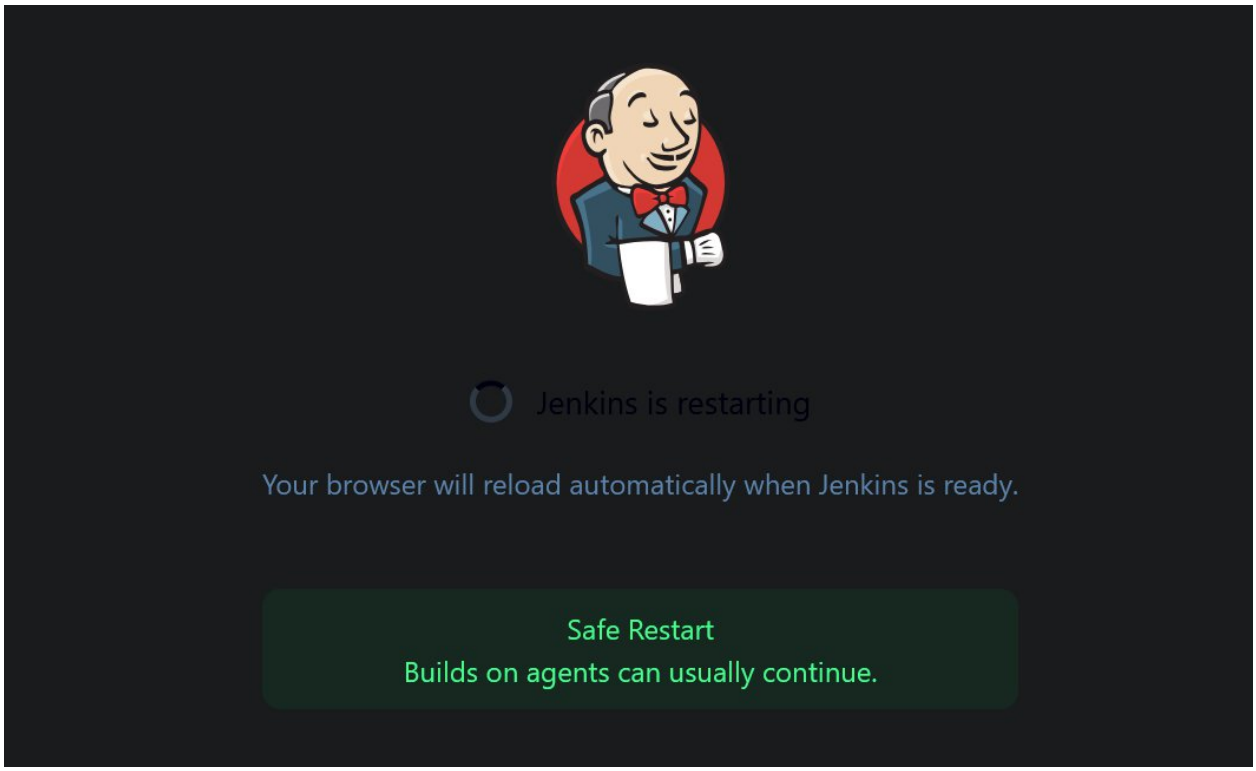
Al definir la directiva agent { docker { image 'php:8.2-cli' } }, Jenkins orquesta la creación de un contenedor efímero en tiempo real. Este contenedor nace exclusivamente para ejecutar la validación de la aplicación de damas, realiza un análisis de sintaxis estática sobre el código base (como el archivo api.php) y, una vez finalizada la tarea, es destruido automáticamente.

Plugin Docker para el Agente Dinámico

Para que Jenkins detectase la sintaxis de docker y poder ejecutar el Docker out of Docker correctamente, instalamos el Plugin “Docker Pipeline” de Jenkins:



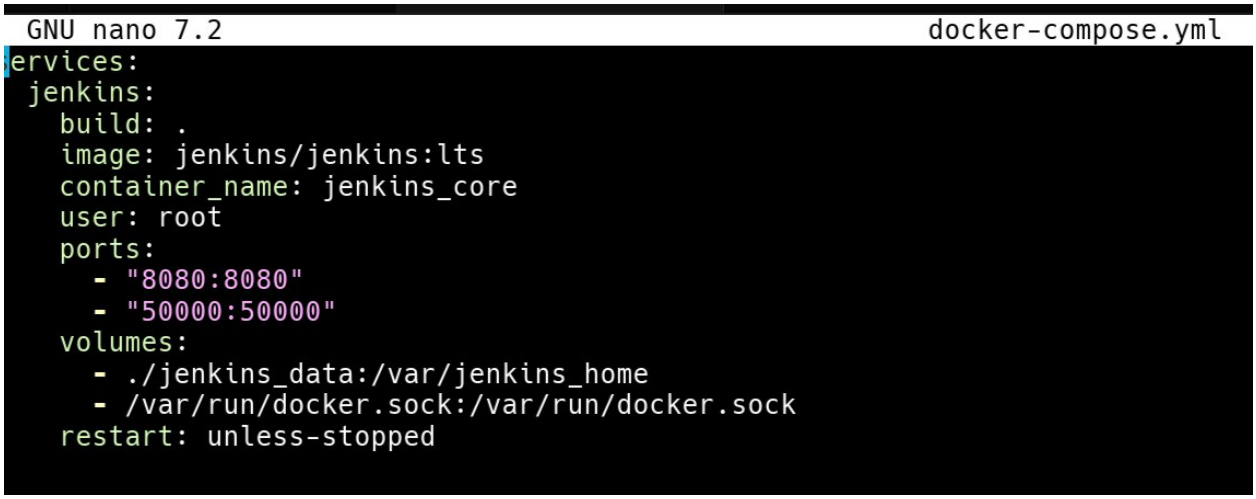
Una vez instalado, se reinicia Jenkins:



Para que Jenkins usase el Plugin de Docker Pipeline tuve que hacer un archivo de configuración Dockerfile dentro de la máquina Ubuntu Server:



Además, al docker-compose.yml (contenedor que alberga Jenkins) le agregué la directiva "build: .":



Primera Pipeline de Integración Continua (CI)

Una vez hecho esto, la primera pipeline de prueba subida a Github se ejecutó correctamente.



Se aprecia al final del Command Output de Jenkins que, como se configuró, se detiene y elimina el contenedor que corre el Agente dinámico.

Una vez validada la etapa de Integración Continua (CI), donde el código fuente se somete a pruebas en un entorno estático con de agentes efímeros, el siguiente hito de la arquitectura es la automatización del Despliegue Continuo (CD).